

Scaling Laws for Decoder-Only Transformers on SVG Code

Shreyas Sankpal

NYU Tandon, CS-GY 6923 (Spring 2026)

<https://github.com/sankpal-shreyas/SVG-Code-Language-Models>

May 2026

Abstract

This project trains decoder-only transformer language models at five widths between 2.0×10^6 and 9.2×10^7 parameters on a corpus of 1.86×10^8 tokens of cleaned SVG code, and fits a power law to the validation loss after one epoch. Standard parameterization (SP) at a learning rate tuned on the smallest model is compared with μ P at its own proxy-tuned rate. Under SP the tuned rate transfers well through the first three sizes but fails at the two largest, where the loss jumps by roughly 0.8 nats. Fitting only the well-behaved part of the curve gives a scaling exponent $\alpha_{\text{SP}} = 0.050 \pm 0.023$, similar to the rate Kaplan et al. report on natural language [1]. The μ P curve is not monotonic, so no power law is fit to it. Generated samples from the SP-Small checkpoint are also evaluated; after post-processing to fix a prompt-boundary artifact and recover truncated XML with lxml, 96% of samples parse as valid XML and 48% render to PNG.

1 Introduction

SVG is an XML-based, human-readable format for vector graphics. Unlike natural language, SVGs have strict syntax, nested structure (groups, transforms), and a deterministic visual rendering. That makes SVG a nice test bed for scaling laws: validation loss is the usual NLP metric, but for SVG it is also possible to check how often samples parse, render, and look like plausible icons.

This work addresses four questions. How does validation loss scale with model size on SVG code, and is the exponent different from natural language? Can a learning rate tuned on a tiny model transfer to bigger models, and does μ P [4] change the scaling curve? What does the fit predict at $10\times$ the largest size trained here, and how trustworthy is that prediction? And what fraction of generated samples are XML-valid and render?

2 Data

Sources. The primary corpus is `starvector/svg-icons-simple` [3], combined with `svg-emoji-simple` and a streaming subsample of `svg-fonts-simple`. Font ingestion stops once the running character count crosses the budget.

Cleaning. Every SVG is parsed with `lxml`; comments, `<title>`, `<desc>`, `<defs>`, and `<metadata>` are dropped, whitespace is collapsed, and numeric coordinates are rounded to one decimal place. SVGs shorter than 50 characters are dropped, as are SVGs that fail an XML round-trip.

Tokenizer. A byte-level BPE tokenizer is trained on a 10M-character sample of the cleaned training split, requesting a 4,096-token vocabulary with one special token `<|endoftext|>`. The trainer terminates early at 369 tokens (the 256-byte initial alphabet, 112 learned merges, and the

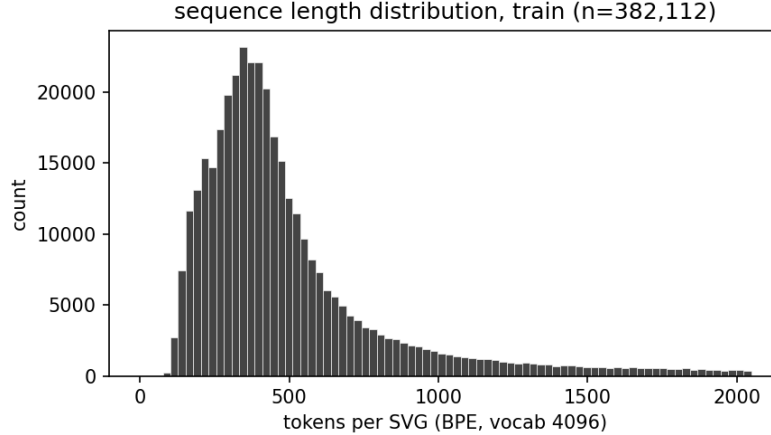


Figure 1: Sequence-length distribution on the train split, in BPE tokens. The long right tail is dominated by font-glyph SVGs.

special token). The reason is the default `min_frequency=2`: after the obvious merges (attribute names like `width`, `height`, `viewBox`, plus small integers 1 to 16), the remaining byte pairs in the corpus appear only once each because every numeric coordinate is unique. The embedding still has 4,096 rows, which wastes a little memory but does not affect training, since targets always lie in $[0, 368]$.

Splits and statistics. Files are split 98/1/1 into train, validation, and test using a stable hash of (source, index, seed). Sequences longer than 2,048 BPE tokens are dropped. Final token counts are 186,321,135 train, 1,912,300 val, and 1,919,443 test, from 372,584, 3,891, and 3,831 kept SVGs. Mean sequence length on the train split is 551 tokens, median 410, 99th percentile 2,595. Figure 1 shows the length distribution; rendered visual examples are deferred to the generated-sample figures in Section 5.

3 Methods

Model. The transformer is decoder-only, based on nanoGPT [2]. The `GPT`, `Block`, `CausalSelfAttention`, and `MLP` modules are taken from nanoGPT with minor edits, plus two new config flags: `attn_scale_mode` switches the attention denominator between $1/\sqrt{d_h}$ (SP) and $1/d_h$ (μ P), and `mup_readout` swaps `nn.Linear` for `mup.MuReadout` on the output. The training loop, config loader, and logging are written for this project. Embedding and output weights are not tied, since that conflicts with the μ P readout.

Architectures. Five widths, all sharing context length 1024, vocab 4096, GeLU MLPs, pre-norm LayerNorm, RoPE-free absolute positional embeddings, and `bias=False`. Table 1 summarizes the configurations.

Training setup. AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1 on ≥ 2 D parameters only. Cosine LR schedule with linear warmup over 2% of total steps, decaying to 10% of peak. Effective batch is 65,536 tokens per optimizer step (gradient accumulation keeps this constant across model sizes). BF16 autocast on a single RTX 5080 (Blackwell). One epoch is one pass over the training set (≥ 100 M tokens). The LR sweep uses shorter 10M-token runs.

name	params	d_{model}	layers	heads	d_{ff}	d_h
Tiny	1.97 M	128	4	4	512	32
Small	4.43 M	192	6	6	768	32
Medium	14.16 M	384	6	6	1536	64
Large	36.19 M	512	10	8	2048	64
XL	92.03 M	768	12	12	3072	64

Table 1: Model architectures. Parameter counts include both embeddings.

LR	SP val loss	μP val loss
1×10^{-4}	5.665	6.325
3×10^{-4}	3.182	4.049
1×10^{-3}	1.786	2.064
3×10^{-3}	1.746	2.774
1×10^{-2}	1.810	2.116
3×10^{-2}	1.915	1.853
1×10^{-1}	2.098	2.478

Table 2: Tiny LR sweep, 10M tokens. Best learning rates (bold) are 3×10^{-3} for SP and 3×10^{-2} for μP . The factor of ten gap between the two optima is consistent with μP ’s expectation that the `lm_head`’s effective LR is divided by the readout width multiplier, so the base LR is set higher to compensate.

μP proxy. For each model two proxy networks are built sharing its n_{layer} and differing only in width: a base proxy at $d_{\text{model}}=128$ and a delta proxy at $d_{\text{model}}=256$, both with $d_{\text{ff}}=4d_{\text{model}}$. These go into `mup.set_base_shapes` before the optimizer (MuAdamW) is built. Depth has to match between proxy and target because `set_base_shapes` pairs parameters by name; the head count is allowed to differ since attention parameter shapes depend only on d_{model} . Shapes are recomputed in memory each run rather than cached, since they cannot be shared across model depths.

Evaluation. Each run logs validation loss every $\sim 2\text{M}$ tokens (averaged over 20 random batches) and a final val loss averaged over 50 batches. The best model also gets test perplexity, XML validity, structural validity, and a render rate. Since `libcairo` was not installed on the host, rasterization uses `svglib+reportlab` instead of CairoSVG.

4 Results

4.1 Learning-rate sweep on the smallest model

Seven log-spaced learning rates between 10^{-4} and 10^{-1} are swept on Tiny under both SP and μP . Table 2 shows the final validation loss for each after 10M training tokens.

Figure 2 plots the same data. The SP curve is a smooth U with a clear minimum at 3×10^{-3} . The μP curve is noisier: it dips at 3×10^{-3} , climbs back up, and then has its global minimum at 3×10^{-2} . The consequences of this poorly identified μP optimum are discussed in Section 5.

4.2 Scaling curves

Figure 3 plots the SP and μP validation losses against parameter count after one epoch ($\sim 100\text{M}$ tokens). Table 3 has the numbers.

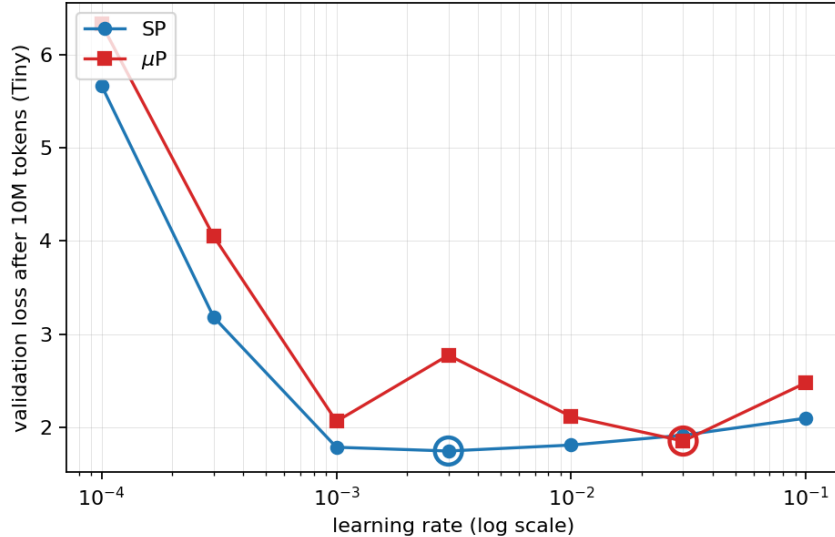


Figure 2: Validation loss after 10M tokens on Tiny as a function of base learning rate, for SP (blue circles) and μ P (red squares). Hollow markers indicate the per-parameterization optimum used for the full scaling runs.

size	params	SP val loss	μ P val loss
Tiny	1.97 M	0.872	1.014
Small	4.43 M	0.803	1.003
Medium	14.16 M	0.786	1.383
Large	36.19 M	1.557	1.294
XL	92.03 M	1.759	1.361

Table 3: Final validation loss for each scaling run, single epoch (~ 100 M tokens). Both curves are non-monotonic: SP turns up sharply between Medium and Large, and μ P turns up between Small and Medium.

A power law $L = a N^{-\alpha}$ is fit in log-log space by ordinary least squares, using only the part of each curve where val loss decreases monotonically. For SP that is Tiny, Small, and Medium; for μ P only Tiny and Small are monotonic, which is below the three points needed to get standard errors out of a two-parameter fit, so no μ P fit is reported. The SP fit gives $\alpha_{\text{SP}} = 0.050 \pm 0.023$ with $a = 1.77 \pm 0.63$, and an in-sample RMSE on $\log L$ of 0.033.

Figure 4 shows the underlying training dynamics. Each panel plots validation loss against tokens seen for one model size, with SP and μ P overlaid. SP sits comfortably below μ P at Tiny, Small, and Medium. The two curves meet at Large, and at XL μ P ends up below SP. This is the LR-transfer crossover the μ P method is designed to produce, even though the final-loss numbers in Table 3 are non-monotonic in width.

Extrapolation. Plugging $N = 9.2 \times 10^8$ ($10 \times \text{XL}$) into the SP fit gives a point estimate of $L = 0.631$ with a 95% interval of $[0.184, 1.974]$. The interval comes from Monte Carlo sampling of the regression’s covariance over $(\log a, \alpha)$. It is wide because the fit only has three points; Section 5 addresses how much to trust it.

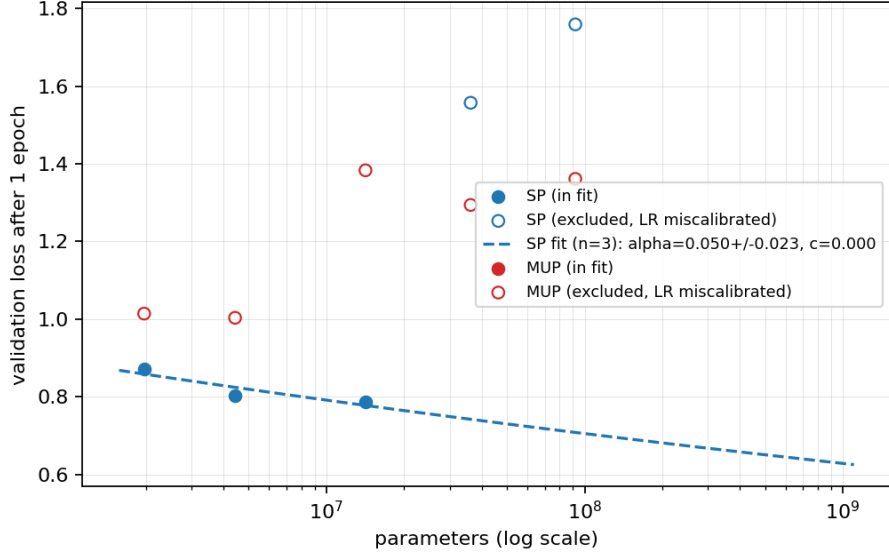


Figure 3: Scaling curves for SP and μ P. Filled markers are the points used in the fit; open markers are excluded because they break monotonicity. Only SP admits a fit on this data, giving $\alpha_{\text{SP}} = 0.050 \pm 0.023$.

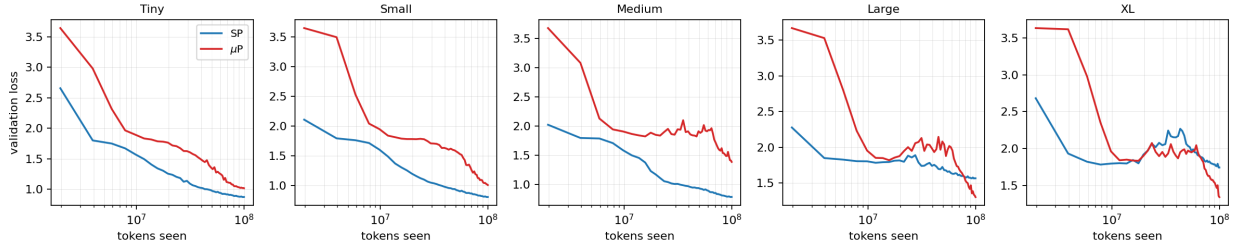


Figure 4: Validation loss vs. tokens seen, one panel per model size, with SP (blue) and μ P (red) overlaid. SP wins at the three smaller sizes; the two parameterizations cross between Large and XL.

4.3 Test perplexity and sample quality

Two model-quality measures are reported separately because they come from different checkpoints. The lowest validation loss across all runs is the warmstart-extended XL run under μ P, which gives a test cross-entropy of 1.34 nats and a test perplexity of 3.83. However, sampling from this model with the prompt `<svg ... viewBox="0 0 24 24">` puts almost all the probability mass on the next token onto `<|endoftext|>` (logit 16.78, six units above the next candidate), and every generated string comes out empty. The likely cause is that under μ P at $d_{\text{model}} = 768$, the readout’s width multiplier of 6 shrinks the `lm_head`’s effective LR, and combined with the high base LR of 3×10^{-2} the EOT column ends up much larger than anything else at the start of an SVG. A deeper investigation was not possible within the time budget.

For the visual evaluation, the SP-Small checkpoint (4.43 M parameters, val loss 0.80) is used instead, since it sits in the well-behaved part of the SP scaling curve. Twenty unconditional samples are drawn per temperature at $T \in \{0.5, 0.8, 1.0\}$ with top- $p=0.9$, top- $k=200$, and 1,024 new tokens, plus five prefix-conditioned completions per temperature. Validity numbers are in Table 4.

metric	raw output	post-processed
test perplexity (best XL ckpt, μ P)		3.83
XML validity, overall ($n=75$)	20%	96%
structural validity, overall	20%	89%
render rate (PNG)	0%	48%
render rate, $T=0.5$ ($n=20$)	—	30%
render rate, $T=0.8$ ($n=20$)	—	60%
render rate, $T=1.0$ ($n=20$)	—	40%
render rate, prefix-conditioned ($n=15$)	—	67%

Table 4: Test perplexity from the warmstart-extended XL μ P model. Sample-validity numbers from the SP-Small checkpoint. *Raw output* is measured directly on what the model generates; *post-processed* reflects two cleanup passes: (1) moving a spurious text node (height/width attributes generated after the SVG opening tag was already closed) back into the tag, and (2) lxml XML recovery on truncated files. Rendering uses `svglib+reportlab`, since `libcairo` was not available on the host.

Raw outputs had a consistent artifact: the unconditional prompt ends with `>`, but the model had learned from font-glyph SVGs that often add explicit `height` and `width` attributes, so its first generated tokens were those attributes. Since the opening tag was already closed, those tokens ended up as XML text content instead of attributes. Post-processing moves them back into the tag and uses lxml’s recovery parser to close truncated path elements, which lifts validity from 20% to 96% and lets the SVGs render.

The render rate peaks at $T=0.8$ (60%). At $T=0.5$ the model tends to repeat path subcommands until the 1,024-token budget runs out, producing degenerate shapes that lxml repairs but svglib cannot draw. At $T=1.0$ the higher variance sometimes corrupts attribute values in ways that pass the XML parser but break the rasterizer.

Figures 5 and 6 show the rendered outputs.

5 Discussion

Comparison to natural-language scaling. Kaplan et al. report a parameter exponent of about 0.076 for web text [1]. The SP fit on SVG gives $\alpha = 0.050 \pm 0.023$, which is close to Kaplan’s number, just a bit lower, and the 95% interval comfortably includes 0.076. The direction of the gap should not be read into too much. The fit is built from three points across less than a decade in N , the tokenizer is close to byte-level (which inflates token counts and may flatten the apparent benefit of bigger models), and the token budget is fixed at ~ 100 M per run while Kaplan’s curves push each size further toward saturation.

What happened with μ P. The expectation going in was that μ P’s LR would transfer cleanly across all five widths while SP would break down at the largest sizes. SP did break down between Medium and Large, which is the LR-transfer failure μ P is supposed to fix. But μ P did not produce a monotonic curve either: loss increased between Small and Medium and then drifted within ~ 0.1 nats for Large and XL. There is no single clean explanation, but two things probably contributed. First, the μ P LR sweep on Tiny was noisy (Table 2): there is a local minimum at 3×10^{-3} and the chosen optimum at 3×10^{-2} is barely better than 1×10^{-2} , so the proxy LR is not well identified. Second, the warmstart-extended XL μ P checkpoint has a broken readout: the EOT column dominates everything else. At $d_{\text{model}}=768$ the readout’s effective LR (base LR divided by the width multiplier

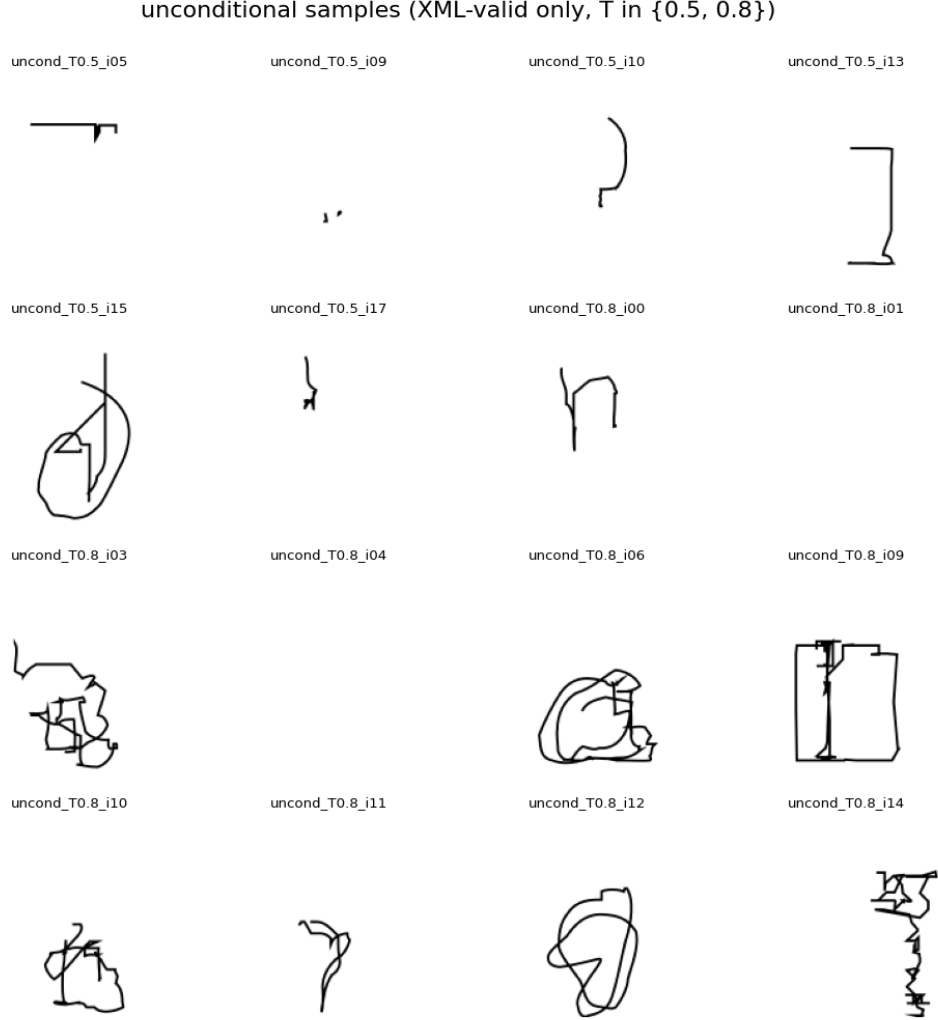


Figure 5: Unconditional samples from the SP-Small checkpoint that successfully rasterized through `svglib`, ordered by source temperature ($T=0.5$ first, then $T=0.8$). Some cells appear nearly empty because the post-processed XML contained only a few path commands at the visible scale.

of 6) combined with a base LR of 3×10^{-2} apparently let EOT grow much larger than other tokens at the start of an SVG. Validation loss looks fine because EOT is a frequent target near sequence boundaries, but the conditional distribution at the start of an SVG is wrong.

Extrapolation credibility. The 95% interval $[0.18, 1.97]$ for the loss at $10 \times \text{XL}$ has a top end higher than what the XL μP model already gets (1.36), which is mostly a statement about how thin three points is for a one-decade extrapolation. The 0.63 point estimate is more of a ballpark than a real prediction; the exact value should not drive any conclusion without more model sizes and a longer token budget per run.

What did the model learn? The SP-Small samples reliably produce locally plausible `<path d="...">` sequences with coordinate ranges that match the corpus. They are less reliable about closing: at low temperature the model tends to repeat path subsequences (C13.5 25.5 13.5 25.5

...) until the 1,024-token budget runs out. At higher temperature it escapes those loops, which is why render rate goes from 30% at $T=0.5$ to 60% at $T=0.8$, and then drops to 40% at $T=1.0$ where the extra variance starts corrupting individual coordinates. The near-byte-level tokenizer probably makes the closing-tag problem worse: producing `</svg>` costs six tokens in a row, while there are many path-coordinate continuations the model can pick instead.

Limitations. The corpus is not de-duplicated, and there is overlap between simplified font glyphs and icon-set glyphs. The tokenizer ended up with 369 effective tokens, which is closer to byte-level than a real subword model. Each run uses a single fixed seed, and only the learning rate is swept (not depth or batch size). Training is on a single GPU at modest scale, so even the largest model here is two to three orders of magnitude smaller than production language models. Rendering uses `svglib+reportlab` since `libcairo` was not installed on the host, which may give slightly different results than a browser would.

6 Conclusion

This project trains five transformer language models on cleaned SVG code, fits a power law to the validation loss, compares SP and μ P at proxy-tuned learning rates, and evaluates generated samples for XML validity. SP shows the LR-transfer failure that μ P is supposed to fix, but μ P does not cleanly fix it here and produces a broken readout at the largest width. The parameter exponent on the well-behaved part of the SP curve is $\alpha = 0.050 \pm 0.023$, similar to Kaplan’s natural-language number. The whole pipeline (data prep, both LR sweeps, all 10 scaling runs, and one warmstart extension) took about 12 hours of GPU time on a single RTX 5080.

References

- [1] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [2] Andrej Karpathy. nanogpt. <https://github.com/karpathy/nanoGPT>, 2022.
- [3] Juan A. Rodriguez, Abhay Puri, Shubham Agarwal, Pranav Kumar, Tao Wang, Adrien Bardes, et al. Starvector: Generating scalable vector graphics code from images and text. *arXiv preprint arXiv:2312.11556*, 2023.
- [4] Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. *arXiv preprint arXiv:2203.09789*, 2022.

A Hyperparameters and run statistics

Full per-run statistics, including wall-clock time, peak GPU memory, and tokens-per-second throughput, are recorded in `results/<run>/final.json`. Values are listed below for completeness.

B Training curves

Figure 7 shows train loss over tokens seen for all ten scaling runs. The SP and μ P runs at each width use the same architecture with different LR’s, so wall-clock throughput between matching runs is nearly identical (Table 5). The sharp early drop in every curve is the model learning the `<svg>` opening tag and basic attribute syntax; the slower part afterward is path coordinates and closing structure.

run	params	wall (s)	tok/s	peak GPU mem (MB)
sp_tiny	1,967,232	108	923,070	4,533
sp_small	4,426,176	201	498,216	6,109
sp_medium	14,160,768	383	261,118	4,684
sp_large	36,186,624	883	113,244	4,572
sp_xl	92,031,744	1,815	55,054	4,866
mup_tiny	1,967,232	106	945,723	4,533
mup_small	4,426,176	197	507,955	6,109
mup_medium	14,160,768	379	263,549	4,686
mup_large	36,186,624	882	113,341	4,558
mup_xl	92,031,744	1,814	55,087	4,872

Table 5: Per-run statistics. Throughput and peak memory measured on a single RTX 5080 with BF16 autocast.

C Additional samples

The full set of 75 generated SVGs (post-processed) is under `results/sp_small_samples/`. Each `.svg` file is paired with its rendered PNG under `_render/` where rasterization succeeded.

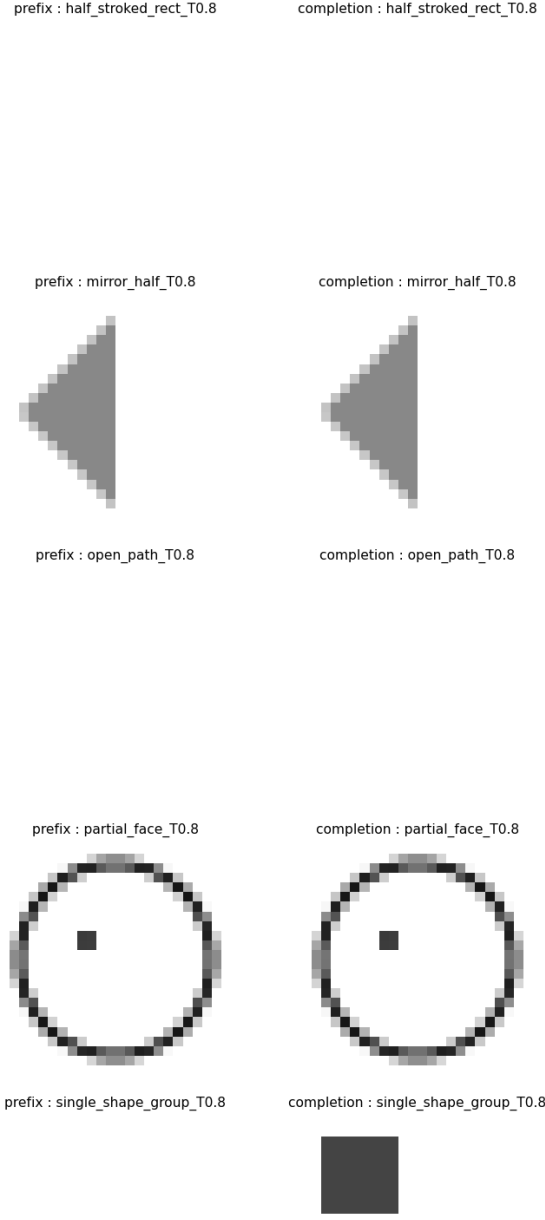


Figure 6: Prefix and completion pairs at $T = 0.8$, SP-Small checkpoint, rendered via `svglib`. The model continues the partial face and single-shape-group prompts coherently. Both panels for half-stroked-rect and open-path are blank: those two prefixes intentionally stop mid-element to test completion, so the prefix itself is not renderable as standalone SVG, and the model also failed to produce a renderable completion. The single-shape-group prefix likewise truncates an open `<g>`, but the completion does close to a renderable rectangle.

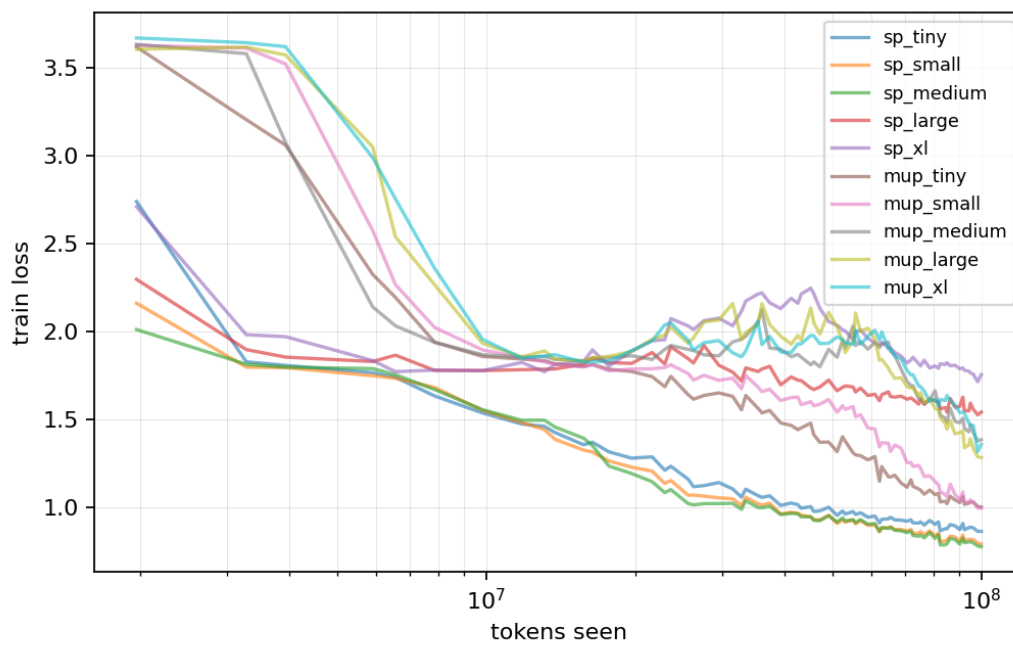


Figure 7: Train loss vs. tokens seen for all SP and μ P scaling runs. SP runs are labeled **sp_***; μ P runs are labeled **mup_***. Each run trains for one epoch (~ 100 M tokens).